

PRACTICAL WEEK 6

1. Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical6.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
GNU nano 8.7.1 practical6.1.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define CLIENT_FIFO "client_fifo"
#define SERVER_FIFO "server_fifo"
#define BUFFER_SIZE 512

int main()
{
    char buffer[BUFFER_SIZE];

    mkfifo(CLIENT_FIFO, 0666);
    mkfifo(SERVER_FIFO, 0666);

    printf("FIFO Server started...\n");
    printf("Waiting for client messages...\n");

    while (1)
    {
        int client_fd = open(CLIENT_FIFO, O_RDONLY);

        if (client_fd == -1)
        {
            perror("open client_fifo");
            exit(EXIT_FAILURE);
        }

        ssize_t bytes_read = read(client_fd, buffer, BUFFER_SIZE - 1);
        close(client_fd);

        if (bytes_read > 0)
        {
            buffer[bytes_read] = '\0';

            printf("Client message: %s\n", buffer);

            char response[BUFFER_SIZE];

            if (strcmp(buffer, "hello") == 0)
            {
                strcpy(response, "Hello Client!");
            }
            else if (strcmp(buffer, "exit") == 0)
```

```

        {
            strcpy(response, "Server shutting down...");
        }
        else
        {
            strcpy(response, "Server processed: ");
            strncat(response, buffer,
                BUFFER_SIZE - strlen(response) - 1);
        }

        int server_fd = open(SERVER_FIFO, O_WRONLY);

        if (server_fd == -1)
        {
            perror("open server_fifo");
            exit(EXIT_FAILURE);
        }

        write(server_fd, response, strlen(response) + 1);
        close(server_fd);

        printf("Response sent: %s\n\n", response);

        if (strcmp(buffer, "exit") == 0)
        {
            break;
        }
    }

    unlink(CLIENT_FIFO);
    unlink(SERVER_FIFO);

    printf("Server terminated.\n");

    return 0;
}

```

Run the code:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical6.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical6.1.c -o practical6.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical6.1

```

OUTPUT:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical6.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical6.1.c -o practical6.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical6.1
FIFO Server started...
Waiting for client messages...

```

Now create a client program:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical6.1_client.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Write the code:

```
GNU nano 8.7.1 practical6.1_client.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define CLIENT_FIFO "client_fifo"
#define SERVER_FIFO "server_fifo"
#define BUFFER_SIZE 256

int main()
{
    char message[BUFFER_SIZE];
    char response[BUFFER_SIZE];

    printf("FIFO Client started.\n");

    while (1)
    {
        printf("Enter message: ");
        fflush(stdout);

        fgets(message, BUFFER_SIZE, stdin);

        message[strcspn(message, "\n")] = '\0';

        int client_fd = open(CLIENT_FIFO, O_WRONLY);

        if (client_fd == -1)
        {
            perror("open client_fifo");
            exit(EXIT_FAILURE);
        }

        write(client_fd, message, strlen(message) + 1);

        close(client_fd);

        int server_fd = open(SERVER_FIFO, O_RDONLY);

        if (server_fd == -1)
        {
            perror("open server_fifo");
            exit(EXIT_FAILURE);
        }

        ssize_t bytes_read =
            read(server_fd, response, BUFFER_SIZE - 1);
```

```

        close(server_fd);
        if (bytes_read > 0)
        {
            response[bytes_read] = '\0';
            printf("Server response: %s\n\n", response);
        }

        if (strcmp(message, "exit") == 0)
        {
            break;
        }
    }

    return 0;
}

```

Run the code:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical6.1_client.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical6.1_client.c -o practical6.1_client
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical6.1_client

```

OUTPUT:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical6.1_client.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical6.1_client.c -o practical6.1_client
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical6.1_client
FIFO Client started.
Enter message: |

```

FINAL OUTPUT:

SERVER:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical6.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical6.1.c -o practical6.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical6.1
FIFO Server started...
Waiting for client messages...
Client message: hi
Response sent: Server processed: hi

Client message: where are you
Response sent: Server processed: where are you

^C
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$

```

CLIENT:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical6.1_client.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical6.1_client.c -o practical6.1_client
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical6.1_client
FIFO Client started.
Enter message: hi
Server response: Server processed: hi

Enter message: where are you
Server response: Server processed: where are you

Enter message: ^C
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

2. Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

Create a file:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical6.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

Write the code:

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

volatile sig_atomic_t sigint_received = 0;
volatile sig_atomic_t sigterm_received = 0;
volatile sig_atomic_t sigusr1_received = 0;

void handle_sigint(int signal)
{
    sigint_received = 1;
}

void handle_sigterm(int signal)
{
    sigterm_received = 1;
}

void handle_sigusr1(int signal)
{
    sigusr1_received = 1;
}

int main()
{
    signal(SIGINT, handle_sigint);
    signal(SIGTERM, handle_sigterm);
    signal(SIGUSR1, handle_sigusr1);

    printf("Signal handling program started.\n");
    printf("Process PID = %d\n", getpid());

    printf("\nSend signals from another terminal:\n");
    printf("SIGINT : kill -SIGINT %d\n", getpid());
    printf("SIGTERM : kill -SIGTERM %d\n", getpid());
    printf("SIGUSR1 : kill -SIGUSR1 %d\n", getpid());

    printf("\nProgram is waiting for signals...\n\n");

    while (1)
    {
        pause();

        if (sigint_received)
        {
            printf("SIGINT received!\n");
            printf("SIGINT is normally generated using Ctrl+C.\n\n");
        }
    }
}

```

```

        sigint_received = 0;
    }

    if (sigusr1_received)
    {
        printf("SIGUSR1 received!\n");
        printf("User-defined signal processed.\n\n");

        sigusr1_received = 0;
    }

    if (sigterm_received)
    {
        printf("SIGTERM received!\n");
        printf("Terminating the program...\n");

        break;
    }
}

printf("Program terminated normally.\n");

return 0;
}

```

Open another terminal:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ kill -SIGUSR1 1390
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ kill -SIGINT 1390
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ kill -SIGTERM 1390
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

OUTPUT:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical6.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical6.2.c -o practical6.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical6.2
Signal handling program started.
Process PID = 1390

Send signals from another terminal:
SIGINT : kill -SIGINT 1390
SIGTERM : kill -SIGTERM 1390
SIGUSR1 : kill -SIGUSR1 1390

Program is waiting for signals...

SIGUSR1 received!
User-defined signal processed.

SIGINT received!
SIGINT is normally generated using Ctrl+C.

SIGTERM received!
Terminating the program...
Program terminated normally.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```